

AI Graph Engineering · Module 1.1 Revision Note

Graph theory you actually need — for automation test engineers

Phase 1 (Foundations) | Prepared for Babatunde | 24 Aug 2026 | Mentor session 1

The one idea that runs through the whole 6 months: almost every hard question in test automation is secretly a question about **connections** — *what do I re-run when this changes? which test is riskiest to break? why do these tests only fail together?* Lists and spreadsheets answer connection questions badly. Graphs answer them natively. That is the entire bet.

1 · Core vocabulary (with a testing example for each)

Term	Plain meaning	Testing example
Node	A thing / entity.	A test, page object, fixture, driver, API endpoint.
Edge	A relationship between two nodes — not an action. Always has two endpoints.	LoginTest —CALLS→ /api/login. The edge is CALLS.
Directed edge	One-way relationship (has an arrow).	Test —DEPENDS_ON→ Fixture (and not the reverse).
Undirected edge	Symmetric relationship (no direction).	TestA —SIMILAR_TO— TestB.
Path	A sequence of edges you can walk.	Test → /api/login → auth-service.
Cycle	A path that loops back to its start.	A → B → A: order-dependent flaky tests waiting on each other.
DAG	Directed Acyclic Graph — directed, no cycles. Can always be put in a valid order (topological sort).	A healthy pipeline, dependency tree, or fixture hierarchy.
Degree	How many edges touch a node.	A page object imported by 300 tests = high degree.
Neighborhood	The nodes one hop away. (Becomes the heart of GNNs in Phase 4.)	Everything directly linked to one test.
Weighted edge	An edge carrying a number / strength.	Test —CALLS→ /checkout weighted by co-failure count.
Self-loop	An edge from a node back to itself.	Test —RETRIES→ Test; recursive fixture setup.
Disconnected component	A node/cluster with no edges to the rest — an island.	An orphaned test or unused fixture = dead-code candidate.

2 · Three classic algorithms → three testing jobs

Algorithm	Asks	Testing job it does
Traversal (BFS/DFS)	What can I reach from here?	Impact analysis / test selection — the tests connected to a change.
Shortest path	What's the closest link between two nodes?	Root-cause tracing between a test and a failing component.
Centrality	Which node is most important?	Risk ranking — find the chokepoints to test first.

3 · The five insights that matter most

- 1. Reachability = impact analysis.** Start at the changed node, walk the dependency edges, collect every test you hit — that set is exactly what to re-run. It replaces the blanket smoke-suite guess with a computed answer. (*Watch the arrow direction: if edges read Test→Component, follow them backward to find impacted tests.*)
- 2. A DAG is a schedule that's actually possible.** No cycles means a valid run order always exists. A cycle means two things each wait on the other — no first step — which is why order-dependent flaky tests are so nasty.

3. **Needing a specific test order = hidden edges between tests**, almost always from shared state. You can either honour those edges (topological sort) or **delete them** (isolate tests with their own fixtures) — and deleting them gives you parallel execution for free. Alphabetical ordering only ever works by accident.
4. **Degree = popularity; betweenness = load-bearing**. For "what breaks the most if it fails," you want betweenness — the chokepoint every journey must route through (e.g. login/auth). A node can have modest degree but huge blast radius.
5. **Disconnected islands are a gift**. A test or fixture with no edges to anything is probably dead code you'd never otherwise spot.

4 · Teach-it-back one-liners (say these to own the concept)

"Nodes are the things; edges are the relationships between things. Two endpoints → edge; stands alone → node."

"A DAG is a to-do list that can be scheduled. A cycle is two tasks each waiting on the other — impossible."

"If tests need a specific order, you have hidden edges. Schedule them (topological sort) or delete them (isolation → parallelism)."

"Degree = how popular a node is. Betweenness = how load-bearing it is. Blast radius follows betweenness."

"Impact analysis = start at what changed, walk the edges, collect every test — just walk them the right direction."

5 · Your revisit list & next step

Reinforce before Module 1.2: (a) **degree vs betweenness** centrality — the one that needed the most work; (b) edge = a *relationship*, not an action; (c) which direction "depends-on" points when doing reachability. **Already solid:** nodes/edges, directed vs undirected, DAGs, reachability = impact analysis, and the login-as-chokepoint intuition. Quiz: ~6/10 on a first pass — a normal, healthy start.

Assignment 1.1 — "Draw your suite" (by hand): model one real Playwright/pytest file as a directed graph (tests, page objects, fixtures, endpoints; verbs USES / DEPENDS_ON / CALLS). In 3 sentences, name the riskiest node to break and how the graph shows it (think chokepoint). Circle any orphaned node. Bring it back for a senior-style review. **Next up:** Module 1.2 — Neo4j & Cypher, where you start *querying* these graphs.